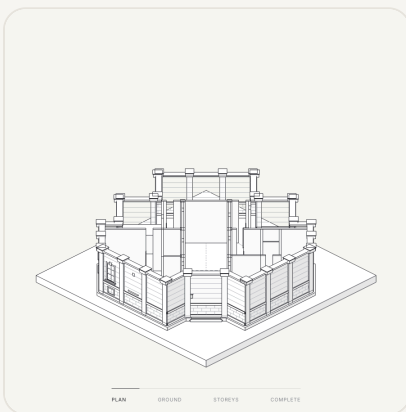
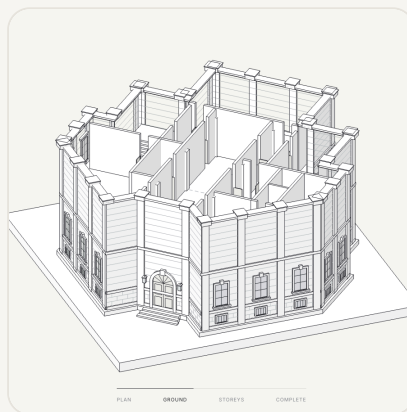


Turn a picture into a *scroll animation.*

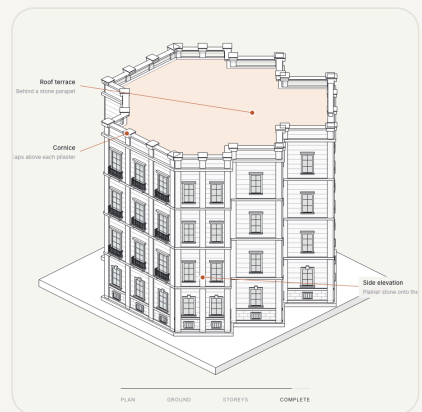
You give AI one picture. It makes five pictures that show your object coming together, then builds a website where the object comes together as you scroll.



START OF THE SCROLL



HALFWAY



END OF THE SCROLL

PROMPT 1

Makes the pictures

Five images of your object, from the first stage to the finished result.

PROMPT 2

Builds the animation

Turns those five pictures into an animation on your website that plays as you scroll.

Five steps. *That's it.*

1

Pick your object

Find one picture of the thing you want to animate: a building, a product, a gadget, anything.

2

Make the five pictures

Open an AI image tool (for example, ChatGPT). Paste **Prompt 1**, change **[YOUR OBJECT]** to your thing, and attach your picture.

Uses Prompt 1

3

Fix any picture you don't like

Not happy with one? Just ask it to redo that picture. Keep all five looking alike: same angle, same size, same colours.

4

Build the animation

Open an AI coding tool (for example, Claude Code) in your website's folder. Paste **Prompt 2** and attach your five pictures. It builds the animation and gives you a link to preview it.

Uses Prompt 2

5

Make it yours

Scroll the preview and ask for changes in plain words, as many times as you like. Try "slower", "zoom in on the details", or "add labels to the parts".

Tip: the better your five pictures match each other, the smoother the animation. Take your time on step 3.

Prompt 1: *make the pictures*

Paste this into your AI image tool. Change **[YOUR OBJECT]** to your object, for example "building construction", and attach your picture.

Create a premium visual concept and five-image transformation sequence for **[YOUR OBJECT]**

PURPOSE

These images will serve as key-pose references for an animation built with Anime.js on my website. Generate images only—no code or website.

CREATIVE DIRECTION

Choose an eye-catching transformation appropriate to the object, with a clear starting state and a satisfying ending state.

Consider unfolding, opening, expanding, rotating, or assembling. Choose the movement that best reveals the object's character and construction. Avoid relying on a simple colour change or fade.

Decide the materials, colour palette, lighting, and background. Use a polished 3D product-render aesthetic with believable depth, refined surfaces, and subtle reflections.

Keep the design striking, readable on a phone, and practical to recreate as separately animated components.

COMPOSITION

Show one centred object against a clean, uniform background.

Choose a fixed camera angle that clearly reveals the transformation.

Use a square canvas with enough space for every stage.

Keep the camera distance, perspective, framing, and object anchor position identical throughout. Do not resize individual poses to fill the frame.

SEQUENCE

Plan the complete object and its movement before generating:

1. Starting state.
2. Transformation approximately 25% complete.
3. Transformation approximately 50% complete.
4. Transformation approximately 75% complete.
5. Ending state.

Make each stage a coherent step in the same transformation. Preserve component identity, count, shape, and materials. Hidden components should appear naturally as they become visible.

Establish the design in the first image, then use it as the visual reference for subsequent images wherever reference-image input is supported.

CONSISTENCY AND CLARITY

Keep colours, textures, background, light positions, and exposure consistent.

Only change component positions and orientations required by the transformation.

Allow shadows and reflections to change naturally with movement.

Keep component boundaries clear and the entire object sharply focused.

Avoid intersecting solid parts, unexplained additions, disappearing components, motion blur, heavy glow, smoke, and particles.

OUTPUT

Deliver five separate high-resolution images with identical dimensions.

Do not combine them into a collage or grid.

No text, labels, logos, watermarks, borders, or unrelated objects.

If only one image can be generated at a time, create the starting frame first and retain the same design specification for the remaining frames.

These are visual references. Do not claim that flattened images contain editable layers or that five key poses alone create a smooth animation.

Prompt 2: *build the animation*

Paste this into your AI coding tool, opened in your website's folder, and attach your five pictures. Nothing needs changing.

Implement a polished, scroll-driven animation on my website using Anime.js, based on the attached object images.

GOAL

Recreate the object and its transformation as a smooth, continuous animation whose progress is driven by the user's scroll position (like the hero on animejs.com). Treat the images as visual references for the starting state, intermediate poses, and ending state.

Inspect my existing website first. Match its framework, layout, and styling, and integrate the animation without replacing unrelated content. If the project is empty, scaffold a minimal Vite project and build a simple hero page around the animation, with at least one section after it.

CHOOSE THE APPROPRIATE APPROACH

Inspect the images and choose the simplest rendering approach that can convincingly reproduce the object:

- Use layered SVG for illustrated objects and transformations that work in two dimensions.
- Use Three.js with Anime.js for objects that require real depth, curved surfaces, changing perspective, or realistic lighting.
- Use separate image layers only if suitable assets exist and their movement does not expose missing surfaces.

Anime.js should control the animation timing and progression.

Do not assume that flattened images contain editable components. Create the necessary SVG shapes or procedural 3D geometry where practical. If essential assets cannot be recreated faithfully, explain the specific limitation before substituting a noticeably different design.

VISUAL FIDELITY

Preserve the reference object's silhouette, proportions, materials, colours, and identifiable components.

Give each moving component its own transform and an appropriate pivot point. Maintain consistent construction throughout the transformation.

Do not substitute a slideshow, crossfade, whole-image zoom, or rotation for the object's actual transformation.

ANIMATION

Build one coordinated, seekable Anime.js timeline.

Use the supplied images as pose guides, interpolating continuously between them. Do not treat the reference images as a complete frame sequence.

Use:

- Natural acceleration and deceleration.
- Subtle staggering between related components.
- Believable hinge, rotation, sliding, or assembly movement.
- A brief hold on the starting pose and on the final reveal.
- Restrained secondary motion where it improves the result.

Keep the camera stable unless the references require movement. Avoid unnecessary bouncing, jitter, abrupt transitions, and intersecting solid parts. Parts that haven't appeared yet must be hidden, not floating in place.

Seeking must be deterministic: jumping to any time, forwards or backwards, must produce exactly the same frame as playing to that time.

SCROLL-DRIVEN PLAYBACK

- Link the timeline to scroll with Anime.js onScroll (ScrollObserver) using smoothed sync (e.g. sync: 0.4), so it eases towards the scroll position instead of stepping with the mouse wheel.
- Place the animation in a tall scroll section (about 300–350vh) with a sticky, full-viewport inner container, so the object stays pinned while scrolling builds it.
- Map the section's scroll range (enter 'top top' → leave 'bottom bottom') to the full build: start pose at the top, finished object at the bottom. Scrolling up must reverse it smoothly.
- In scroll mode the timeline plays the build once and does not loop.
- On desktop, pin the hero copy beside the animation. On mobile, let the copy scroll normally, then pin the animation.
- Show a small progress indicator with clickable stage labels that smooth-scroll to each stage, and a "scroll" hint that fades out once scrolling starts.
- Release the pin cleanly into the following content when the build completes.
- Keep an optional autoplay mode (with a seamless loop that returns to the starting pose, matching position and motion at the boundary) for use without scroll.

WEBSITE INTEGRATION

Create a reusable, responsive component with a small API (play, pause, seek, destroy, and a scroll option). Keep animation styles scoped to the component. Avoid global CSS changes and duplicate dependencies.

Handle resizing (including refreshing the scroll observer), pause unnecessary work when offscreen or when the tab is hidden, and clean up timelines, scroll observers, listeners, and rendering resources when the component is removed.

Respect prefers-reduced-motion: remove the tall scroll section, show a strong static pose of the finished object with an optional play control, and switch modes if the preference changes.

PERFORMANCE

Aim for stable 60 fps on capable devices, including while scrolling, without promising it universally. Render only when the timeline changes. Keep geometry, effects, and animated elements economical. For 3D, cap rendering resolution appropriately for mobile devices.

Load required assets before starting so the object does not appear in incomplete pieces.

VERIFICATION AND DELIVERY

Implement the working animation, not just an explanation.

Run it in the browser and check:

- Visual similarity to the supplied references.
- Smooth movement through intermediate poses.
- Scroll position maps correctly to progress (e.g. 60% through the section = 60% built), smoothing works, and scrolling up reverses cleanly.
- Backward seeks produce the same frames as forward playback.
- Progress UI stays correct during holds where nothing moves.
- The pin releases correctly into the next section.
- Autoplay loop behaviour, if included.
- Desktop and mobile layout, with no horizontal scroll.
- Reduced-motion behaviour.
- Browser errors and integration issues.

Provide a working preview and identify the files changed. Briefly explain how to adjust scroll length, smoothing, duration, colours, and component size.

Clearly state any visual compromises. Do not claim that performance or behaviour was tested unless it actually was.